

Ingeniería del Software de Componentes y Sistemas Distribuidos

PRUEBA DE EVALUACIÓN CONTINUA – PEC 2

Presentación

Esta PEC 2 y última prueba del curso recoge algunas de las buenas prácticas en el desarrollo y puesta en producción del software en arquitecturas distribuidas. Se trabajan algunos de los conceptos clave durante el desarrollo y puesta en producción, como son la entrega continua, la cultura DevOps y el despliegue utilizando contenedores.

Para elaborar la PEC se formulan cuatro ejercicios: 1) evaluar un aspecto a concretar de los principios fundamentales de la cultura DevOps; 2) razonar sobre los aspectos clave de las pruebas en la entrega continua de software de calidad; 3) evaluar uno de los aspectos clave de los contenedores y plataformas de orquestación; 4) explicar la construcción de un “pipeline” de entrega continua para probar y desplegar los microservicios implementados y testeados a las prácticas anteriores.

La PEC 2 abarca los contenidos de los libros "Microservices Patterns", "Continuous Delivery" y "The DevOps Handbook" (ver en el apartado Recursos los capítulos que hay que estudiar).

Competencias

En esta PEC 2 se trabaja las siguientes competencias del Grado en Ingeniería Informática:

- Saber construir aplicaciones informáticas mediante técnicas de desarrollo, integración y reutilización.
- Aplicación de las técnicas específicas de la Ingeniería del Software a las diferentes etapas del ciclo de vida de un proyecto.

Objetivos

Los objetivos de la PEC 2 son:

- Reconocer los factores de entrega, despliegue e integración continua.
- Interpretar la cultura ágil en el desarrollo de software.
- Entender los “pipelines” de entrega, despliegue e integración continua.
- Exponer argumentos sobre las ventajas y costes de los contenedores y plataformas de orquestación.

Descripción de la PEC 2 a realizar

Ejercicio 1 (2 puntos)

Al inicio de esta actividad os hemos propuesto el siguiente debate: “Debatir sobre los objetivos básicos que persigue una cultura DevOps y los mecanismos, herramientas y/o estrategias para conseguirlos”. En este ejercicio os pedimos que escojáis uno de los objetivos que hayáis debatido en el espacio Debate del aula para explicar cómo lo aplicarías al proyecto **CultureInDaHouse**.

Solución

En el debate del aula han salido muchos objetivos que persigue la cultura DevOps, entre otros:

- Aumentar la flexibilidad.
- Reducir o eliminar los conflictos entre los departamentos de operaciones y desarrollo.
- Automatizar al máximo los procesos de comunicación entre los departamentos de desarrollo y los de operaciones.
- Agilizar las entregas y la introducción de cambios en producción, consiguiendo una frecuencia de entregas más alta.
- Establecer un sistema de desarrollo y entrega más eficiente que permita reducir costes.
- Reducir los errores en entorno de producción y mejorar así la calidad del producto desarrollado
- Abreviar el ciclo de vida del desarrollo de software
- Proporcionar una entrega continua de software de alta calidad.
- Establecer mecanismos óptimos y fluidos de comunicación entre departamentos para aliviar y transparentar todo lo posible las colaboraciones entre todas las áreas.

Cualquiera de estos objetivos con una explicación clara, concreta, concisa, bien redactada y estructurada, y con un ejemplo de cómo lo haría para conseguirlo en el marco del proyecto **CultureInDaHouse**, se considera como solución correcta al ejercicio.

Ejercicio 2 (2 puntos)

En la PRAC3 desarrollásteis un conjunto de pruebas unitarias, usando Junit, Spring Boot Test, Mockito y AssertJ para el microservicio *Show Catalog* de **CultureInDahouse**. Responde brevemente a las siguientes cuestiones:

1. ¿Creéis que son importantes estas pruebas unitarias para conseguir llevar nuevas funcionalidades a producción de forma rápida? Razonad la respuesta.

2. En la PRAC3 desarrollásteis las pruebas unitarias una vez ya teníais los servicios desarrollados, ¿creéis que es la mejor alternativa? Razonad la respuesta.

Solución

1. Las pruebas unitarias son clave en el desarrollo de software de calidad, concretamente en la rápida entrega de nuevas funcionalidades. La entrega continua de software se basa en la construcción, prueba y despliegue automatizado de software y uno de los mecanismos para garantizar que el software que se despliega no tenga errores es con una buena batería de test unitarios que se ejecuten como parte del *pipeline* de integración continua, que proporcionen una elevada cobertura y aborten la construcción y posterior despliegue en caso de errores. Estos tests unitarios no son los únicos que se tienen que pasar, pero sí son una parte importante en el momento de garantizar que las nuevas funcionalidades no introducen errores de regresión o de cualquier otro tipo.
2. Una mejor alternativa podría ser realizar las pruebas unitarias a medida que se van desarrollando las funcionalidades como parte integral del desarrollo de las mismas. Hacerlo así permite tener siempre una buena batería de test unitarios ya que son parte del desarrollo de cada funcionalidad.

También se puede ir un paso más allá y desarrollar, para cada funcionalidad, primero los tests que la prueban y después el código que la implementa. Esta técnica es conocida como *Test Driven Development* y es ampliamente utilizada en proyectos ágiles de desarrollo de software.

Ejercicio 3 (2 puntos)

Una posible estrategia de despliegue para el microservicio *Show Catalog* de **CultureInDaHouse** es el uso de contenedores Docker y de Kubernetes como plataforma de orquestación. Enumerad las ventajas e inconvenientes de esta estrategia, y comentad si creéis que es una buena elección.

Solución

Ver “*Chapter 12. Deploying microservices*” de la bibliografía “*Microservices Patterns*”. Concretamente los apartados “*12.3. Deploying services using the Service as a container pattern*” y “*12.4. Deploying the FTGO application with Kubernetes*”.

Lo mismo que se explica para la aplicación FTGO de ejemplo es aplicable al microservicio *Show Catalog* de **CultureInDaHouse**.

Ejercicio 4 (4 puntos)

Explicad los pasos que habría que hacer en el proyecto **CultureInDaHouse** para conseguir Continuous Deployment. Podéis explicarlo con herramientas concretas o bien con pasos agnósticos a la tecnología/herramientas empleadas.

Solución

Con el fin de conseguir *Continuous Deployment* es necesario que construyamos un *pipeline* automatizado que permita, de forma muy resumida y simplificada, instalar automáticamente una versión del software a producción cada vez que los desarrolladores integren una nueva funcionalidad en el código fuente.

Los pasos del *pipeline* a construir, de forma muy general, serían los siguientes:

- Construcción del proyecto. La construcción del proyecto se lanza automáticamente a partir de la subida de nuevas funcionalidades al repositorio de código fuente.
- Ejecución de los distintos tipos de test (unitarios, integración, etc...). El *pipeline* normalmente está estructurado en *Quality Gates* que indican si la *release* puede seguir adelante o se aborta la construcción.
- Despliegue automatizado del software a los diferentes entornos. Para conseguir *Continuous Delivery* es necesario que el software se despliegue automáticamente en cada entorno.

Los pasos concretos para conseguir cada *pipeline* de *Continuous Delivery* son diferentes en función de la plataforma/herramienta utilizada (GitHub, GitLab, Jenkins, etc...). En esta solución hemos mostrado un conjunto de pasos muy generales y agnósticos de la herramienta utilizada.

Se considerarán correctas todas las soluciones que hayan indicado los pasos concretos centrados en una herramienta concreta.

Recursos

Recursos Básicos

- Libro "Microservices Patterns" (Chris Richardson) Chapter 12. Introduction.
- Libro "Microservices Patterns" (Chris Richardson) Chapter 12 "Deploying microservices".
- Libro "Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation" (Jez Humble & Dave Farley) Parte I: Chapters 1-10.
- Libro "The DevOps Handbook" (Patrick Debois; John Willis; Jez Humble; Gene Kim): Chapters 1 - 4.

Criterios de evaluación

- La PEC 2 se tiene que resolver de forma **estrictamente individual**. En caso de detectar copias se penalizará la actividad con una D como nota.
- El peso de cada ejercicio está indicado en el enunciado.

- Es necesario justificar la solución a cada uno de los ejercicios. Se valorará tanto la corrección de la solución como la justificación dada.

Formato y fecha de entrega

Hay que entregar un único documento PDF con las respuestas a todos los ejercicios. El nombre del archivo tiene que ser: *ApellidosNombre_ISCSD_PEC2.pdf*.

Este documento se entregará en el espacio de Registro de Evaluación Continuada (REC) del aula antes de las **23:59 horas del día 24 de junio de 2022**. No se aceptarán entregas fuera de plazo.